

Prisma – Comandos Essenciais

1) Setup básico

```
# Inicializa o Prisma no projeto
npx prisma init

# Inicializa já definindo o provider
npx prisma init --datasource-provider postgresql
```

Dica: configure `DATABASE_URL` no `.env` após o `init`.

2) Geração do Client

```
# Gera o @prisma/client a partir do schema
npx prisma generate
```

Rode **sempre** após mudar `schema.prisma`.

3) Migrations (fluxo recomendado)

```
# Cria e aplica uma migration em DEV (pede/recebe um nome)
npx prisma migrate dev --name <nome-da-migration>

# Mostra o status das migrations
npx prisma migrate status

# Aplica migrations existentes (PROD/CI)
npx prisma migrate deploy

# Reseta o banco em DEV (drop + reaplica migrations)
npx prisma migrate reset
```

4) Sincronizar sem migration (apenas DEV)

```
# Empurra o schema pro banco sem criar migration
npx prisma db push
```

5) Trabalhar com banco existente (reverse)

```
# Lê o esquema do banco e atualiza o schema.prisma  
npx prisma db pull
```

6) Seed (popular o banco)

```
# Executa o script de seed (defina em package.json > prisma.seed)  
npx prisma db seed
```

Exemplo no `package.json`:

```
{  
  "prisma": {  
    "seed": "ts-node prisma/seed.ts"  
  }  
}
```

7) Studio (GUI para CRUD rápido)

```
# Abre o Prisma Studio  
npx prisma studio
```

8) Validação & Formatação do schema

```
# Valida o schema.prisma  
npx prisma validate  
  
# Formata o schema.prisma  
npx prisma format
```

9) Utilidades

```
# Versões (CLI, client, engines)  
npx prisma -v
```

```
# Ajuda geral / específica\ npx prisma --help
npx prisma migrate --help
```

10) Fluxos comuns

Projeto novo (DEV)

```
npx prisma init
# edite o schema.prisma
npx prisma migrate dev --name init
npx prisma generate
npx prisma studio
```

Já existe um banco com tabelas

```
npx prisma init
# configure DATABASE_URL no .env
npx prisma db pull
npx prisma generate
```

Teste rápido sem migration

```
npx prisma db push
```

Produção/CI (apenas aplicar)

```
npx prisma migrate deploy
```

11) Scripts úteis (package.json)

```
{
  "scripts": {
    "prisma:gen": "prisma generate",
    "prisma:dev": "prisma migrate dev",
    "prisma:deploy": "prisma migrate deploy",
    "prisma:reset": "prisma migrate reset -f",
    "prisma:push": "prisma db push",
    "prisma:pull": "prisma db pull",
    "prisma:studio": "prisma studio",
    "prisma:fmt": "prisma format",
    "prisma:validate": "prisma validate"
  }
}
```

```
}  
}
```

12) Erros comuns & Dicas

- **P1001 – Can't reach database server:** verifique `DATABASE_URL`, host/porta, container do DB e rede.
- **P2002 – Unique constraint failed:** conflito em campo único (`email`, `username`, etc.). Trate no código.
- Use `select` no `prisma.*.create/find*` para **não devolver** em respostas.
- Após alterar o schema, rode `migrate dev` (ou `db push` em DEV) e `generate`.

13) Snippets úteis

create com select (omitindo senha):

```
await prisma.user.create({  
  data: { /* ... */ },  
  select: {  
    id: true, name: true, email: true, username: true,  
    profile_image_url: true, bio: true, is_admin: true, premium: true,  
    created_at: true, updated_at: true,  
    // password_hash: false // (não incluir já omite)  
  },  
});
```

db push com preview (sem aplicar):

```
npx prisma db push --preview-featur  
e
```